

ROTEIRO DE EXPERIMENTAÇÃO PRÁTICA

Implementação, Benchmark e Visualização de Dados O-RAN em
Kubernetes

Mariana Siano

Dezembro de 2025

Sumário

1	Introdução	3
1.1	Ambiente	3
1.2	Pré-Requisitos	3
2	FASE 1: Instalação da Base (<i>Docker</i>)	4
2.1	Como Ligar o <i>Docker</i> (Obrigatório)	4
3	FASE 2: Instalação do Orquestrador (<i>Kind</i>)	5
3.1	Instalação do <i>Metrics Server</i> (Crucial)	5
4	FASE 3: Desenvolvimento dos Códigos	6
4.1	1. O Servidor Central: <i>ric_mestre.py</i>	6
4.2	2. O <i>xApp</i> Funcional: <i>xapp_mock.py</i>	7
4.3	3. O <i>Worker</i> de <i>Benchmark</i> : <i>app_worker.py</i>	7
4.4	4. O <i>rApp</i> : <i>rapp_mock.py</i>	8
4.5	Criar o <i>Dockerfile</i>	9
5	FASE 4: Montagem e <i>Deploy</i>	10
5.1	Construir Imagens	10
5.2	Criar Arquitetura (<i>oran-architecture.yaml</i>)	10
5.3	Script de <i>Benchmark</i> (<i>benchmark_worker.sh</i>)	12
6	FASE 5: Visualização Gráfica (<i>Python</i>)	14
6.1	1. Instalar Bibliotecas	14
6.2	2. Scripts de Gráficos (<i>gerar_graficos.py</i>)	14
7	Rotina Diária de Execução	16
7.1	Passo 1: Ligar o <i>Docker</i>	16
7.2	Passo 2: Preparar o <i>Cluster</i>	16
7.3	Passo 3: Verificar o Status Geral	16
7.4	Passo 4: Atualizar o Código no <i>Kubernetes</i>	16
7.5	Passo 5: Rodar o Benchmark	16

7.6	Passo 6: Gerar os Gráficos	16
8	Referências	17

1 Introdução

Este documento consiste em um roteiro técnico passo a passo detalhado para a configuração de um laboratório *O-RAN* completo. O objetivo desta prática é implementar interfaces de comunicação *A1* e *A2*, executar um ***Benchmark Científico*** e gerar gráficos de desempenho escalando de 1 a 128 microsserviços.

1.1 Ambiente

Sistema Operacional: Ubuntu 22.04 LTS no VirtualBox

1.2 Pré-Requisitos

- **RAM:** 8 GB (Recomendado) ou 4 GB (Mínimo);
- **CPUs:** 3 vCPUs;
- **Disco:** 55 GB.

2 FASE 1: Instalação da Base (*Docker*)

Execute este bloco exato para instalar o *Docker* corretamente

Se já fez, pule esta etapa, mas lembre-se de ligar o serviço.

```
1 # 1. Baixar chaves de segurança e dependências
2 sudo apt-get update
3 sudo apt-get install -y ca-certificates curl gnupg lsb-release
4 sudo mkdir -p /etc/apt/keyrings
5 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
   dearmor -o /etc/apt/keyrings/docker.gpg
6
7 # 2. Adicionar o Repositorio Oficial do Docker
8 echo \
9     "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/
   docker.gpg] https://download.docker.com/linux/ubuntu \
10    $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.
   list > /dev/null
11
12 # 3. Atualizar a lista (Agora ele vai achar o Docker)
13 sudo apt-get update
14
15 # 4. Instalar o Docker agora
16 sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
   compose-plugin
17
18 # 5. Adicionar seu usuario ao grupo do Docker e ativar
19 sudo usermod -aG docker $USER
20 newgrp docker
```

2.1 Como Ligar o *Docker* (Obrigatório)

Sempre que ligar a máquina virtual, garanta que o *Docker* está rodando com este comando:

```
1 sudo systemctl start docker
```

3 FASE 2: Instalação do Orquestrador (*Kind*)

Se já instalou o *Kind* e o *Kubectkl*, pule para a instalação do *Metrics Server*.

```
1 curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/
  release/stable.txt)/bin/linux/amd64/kubectkl"
2 chmod +x kubectkl && sudo mv kubectkl /usr/local/bin/
3
4 [ $(uname -m) = x86_64 ] && curl -Lo ./kind https://kind.sigs.k8s.io/dl/
  v0.20.0/kind-linux-amd64
5 chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind
```

3.1 Instalação do *Metrics Server* (Crucial)

Isso é obrigatório para que o teste de *CPU* e *RAM* funcione. Se não fizer isso, o resultado será vazio.

```
1 # 1. Primeiro, criar um cluester:
2 kind create cluster --name oran-lab
3
4 # 2. Baixar o manifesto oficial
5 kubectkl apply -f https://github.com/kubernetes-sigs/metrics-server/
  releases/latest/download/components.yaml
6
7 # 3. Aplicar corre o para funcionar no Kind (Lab Environment)
8 kubectkl patch deployment metrics-server -n kube-system --type='json' -p=
  '[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "
  value": "--kubelet-insecure-tls"}]'
9
10 # 4. Verificar se esta rodando (Deve aparecer "1/1" ou "Ready")
11 kubectkl get deployment metrics-server -n kube-system
```

4 FASE 3: Desenvolvimento dos Códigos

Vamos criar os scripts que simulam o *RIC*, os *xApps* e o *rApps* usando comunicação *JSON*.

Local: Pasta Downloads/ORAN_LAB

```
1 mkdir -p ~/Downloads/ORAN_LAB
2 cd ~/Downloads/ORAN_LAB
```

4.1 1. O Servidor Central: ric_mestre.py

Este código simula o *Near-RT RIC*. Inclui Interfaces *A1/E2* e a correção do `send_header`.

```
1 cat <<EOF > ric_mestre.py
2 import http.server
3 import socketserver
4 import random
5 import json
6 import threading
7
8 PORT = 8080
9 POLICY_ENERGY_SAVING = False
10
11 class ThreadingHTTPServer(socketserver.ThreadingMixIn, socketserver.
    TCPServer):
12     daemon_threads = True
13
14 class TrafficHandler(http.server.SimpleHTTPRequestHandler):
15     def log_message(self, format, *args):
16         return
17
18     def do_GET(self):
19         global POLICY_ENERGY_SAVING
20         capacity = 80 if POLICY_ENERGY_SAVING else 100
21
22         response = {
23             "load": random.randint(50, 150),
24             "capacity": capacity,
25             "policy_active": POLICY_ENERGY_SAVING
26         }
27         self.send_response(200)
28         self.send_header('Content-type', 'application/json')
29         self.end_headers()
30         self.wfile.write(json.dumps(response).encode())
31
32     def do_PUT(self):
33         global POLICY_ENERGY_SAVING
34         length = int(self.headers['Content-Length'])
35         data = json.loads(self.rfile.read(length).decode())
36
37         if data.get('type') == 'A1_POLICY':
38             POLICY_ENERGY_SAVING = data.get('value', False)
39             print(f"[INTERFACE A1] Nova Politica Recebida: Economia={
POLICY_ENERGY_SAVING}")
40
41         self.send_response(200)
42         self.end_headers()
```

```

43
44     def do_POST(self):
45         length = int(self.headers['Content-Length'])
46         data = json.loads(self.rfile.read(length).decode())
47
48         if 'latency_ms' in data:
49             print(f"[DADO_CIENTIFICO] App:{data['id']} | Latencia:{data
50             ['latency_ms']:.2f}ms")
51
52         self.send_response(200)
53         self.end_headers()
54
55     print(f"--- RIC MESTRE RODANDO (PORTA {PORT}) ---")
56     print(f"--- Interfaces A1 (PUT) e E2 (GET/POST) Ativas ---")
57     with ThreadingHTTPServer(("", PORT), TrafficHandler) as httpd:
58         httpd.serve_forever()
59 EOF

```

4.2 2. O *xApp* Funcional: `xapp_mock.py`

Simula um aplicativo de *Traffic Steering*.

```

1 cat <<EOF > xapp_mock.py
2 import time
3 import os
4 import urllib.request
5 import json
6
7 ric_host = os.getenv('RIC_HOST', 'localhost')
8 url = f"http://{ric_host}:8080"
9 print(f"--- xApp TRAFFIC STEERING INICIADO ---")
10
11 while True:
12     try:
13         with urllib.request.urlopen(url, timeout=2) as response:
14             data = json.loads(response.read().decode())
15
16             util = (data['load'] / data['capacity']) * 100
17             decision = "HANDOVER" if util > 100 else "KEEP"
18
19             report = {"id": "xapp-traffic", "decision": decision}
20             data_bytes = json.dumps(report).encode('utf-8')
21             req = urllib.request.Request(url, data=data_bytes, method="POST"
22             )
23             req.add_header('Content-Type', 'application/json')
24             with urllib.request.urlopen(req) as r: pass
25
26     except: pass
27     time.sleep(2)
28 EOF

```

4.3 3. O *Worker* de *Benchmark*: `app_worker.py`

Focado exclusivamente em medir a latência científica.

```

1 cat <<EOF > app_worker.py

```

```

2 import time
3 import os
4 import urllib.request
5 import json
6 import socket
7
8 ric_host = os.getenv('RIC_HOST', 'localhost')
9 my_id = socket.gethostname()
10 url = f"http://{ric_host}:8080"
11
12 print(f"--- APP WORKER BENCHMARK ({my_id}) ---")
13
14 while True:
15     try:
16         start_time = time.time() # T0
17
18         with urllib.request.urlopen(url, timeout=2) as response:
19             data = json.loads(response.read().decode())
20
21         latency_ms = (time.time() - start_time) * 1000 # T1 - T0
22
23         report = {
24             "id": my_id,
25             "latency_ms": latency_ms,
26             "type": "benchmark"
27         }
28
29         data_bytes = json.dumps(report).encode('utf-8')
30         req = urllib.request.Request(url, data=data_bytes, method="POST"
31     )
32         req.add_header('Content-Type', 'application/json')
33
34         with urllib.request.urlopen(req, timeout=2) as r: pass
35
36     except Exception as e:
37         print(f"Erro: {e}")
38
39     time.sleep(1) # Frequencia 1Hz
40 EOF

```

4.4 4. O *rApp*: rapp_mock.py

Simula o *Non-RT RIC* via Interface A1

```

1 cat <<EOF > rapp_mock.py
2 import time
3 import os
4 import urllib.request
5 import json
6
7 ric_host = os.getenv('RIC_HOST', 'localhost')
8 url = f"http://{ric_host}:8080"
9
10 print(f"--- rApp ENERGY SAVER (Non-RT) INICIADO ---")
11 print(f"--- Conectando via Interface A1 em {url} ---")
12 state = False
13

```



```

14 while True:
15     try:
16         state = not state
17         policy = {
18             "type": "A1_POLICY",
19             "scope": "UE_GROUP_1",
20             "value": state
21         }
22
23         data_bytes = json.dumps(policy).encode('utf-8')
24         req = urllib.request.Request(url, data=data_bytes, method="PUT")
25         req.add_header('Content-Type', 'application/json')
26
27         with urllib.request.urlopen(req) as r:
28             print(f"[rApp] Enviado A1 Policy: Economia={state}")
29
30     except Exception as e:
31         print(f"Erro ao contatar RIC: {e}")
32
33     time.sleep(15)
34 EOF

```

4.5 Criar o *Dockerfile*

Empacotamos todos os scripts em uma imagem unificada.

```

1 cat <<EOF > Dockerfile
2 FROM python:3.9-slim
3 WORKDIR /app
4 COPY ric_mestre.py xapp_mock.py app_worker.py rapp_mock.py ./
5 CMD ["python", "-u", "ric_mestre.py"]
6 EOF

```

5 FASE 4: Montagem e *Deploy*

5.1 Construir Imagens

```
1 cd ~/Downloads/ORAN_LAB
2 docker build -t oran-lab:v1 -f Dockerfile .
```

5.2 Criar Arquitetura (oran-architecture.yaml)

```
1 cat <<EOF > oran-architecture.yaml
2 apiVersion: v1
3 kind: Service
4 metadata:
5   name: ric-service
6 spec:
7   selector:
8     app: ric-platform
9   ports:
10    - protocol: TCP
11      port: 8080
12      targetPort: 8080
13 ---
14 apiVersion: apps/v1
15 kind: Deployment
16 metadata:
17   name: ric-platform
18 spec:
19   replicas: 1
20   selector:
21     matchLabels:
22       app: ric-platform
23   template:
24     metadata:
25       labels:
26         app: ric-platform
27     spec:
28       containers:
29       - name: ric
30         image: oran-lab:v1
31         imagePullPolicy: IfNotPresent
32         command: ["python", "-u", "ric_mestre.py"]
33         resources:
34           requests:
35             cpu: "100m"
36             memory: "100Mi"
37 ---
38 apiVersion: apps/v1
39 kind: Deployment
40 metadata:
41   name: xapp-traffic
42 spec:
43   replicas: 1
44   selector:
45     matchLabels:
46       app: xapp-functional
47   template:
```

```

48     metadata:
49         labels:
50             app: xapp-functional
51     spec:
52         containers:
53         - name: xapp
54           image: oran-lab:v1
55           imagePullPolicy: IfNotPresent
56           command: ["python", "-u", "xapp_mock.py"]
57           env:
58             - name: RIC_HOST
59               value: "ric-service"
60 ---
61 apiVersion: apps/v1
62 kind: Deployment
63 metadata:
64     name: rapp-energy
65 spec:
66     replicas: 1
67     selector:
68         matchLabels:
69             app: rapp
70     template:
71         metadata:
72             labels:
73                 app: rapp
74         spec:
75             containers:
76             - name: rapp
77               image: oran-lab:v1
78               imagePullPolicy: IfNotPresent
79               command: ["python", "-u", "rapp_mock.py"]
80 ---
81 apiVersion: apps/v1
82 kind: Deployment
83 metadata:
84     name: benchmark-worker
85 spec:
86     replicas: 1
87     selector:
88         matchLabels:
89             app: bench-worker
90     template:
91         metadata:
92             labels:
93                 app: bench-worker
94         spec:
95             containers:
96             - name: worker
97               image: oran-lab:v1
98               imagePullPolicy: IfNotPresent
99               command: ["python", "-u", "app_worker.py"]
100             env:
101             - name: RIC_HOST
102               value: "ric-service"
103             resources:
104                 limits:
105                     cpu: "50m"

```

```

106         memory: "50Mi"
107 EOF

```

5.3 Script de *Benchmark* (benchmark_worker.sh)

Este script automatiza o teste, corrige o *Metrics Server* e salva os dados em resultados.csv.

```

1 cat <<EOF > benchmark_worker.sh
2 #!/bin/bash
3 echo "workers,cpu_total_m,ram_total_mi,latencia_media_ms" > resultados.
  csv
4 echo "=== INICIANDO BENCHMARK CIENTIFICO (1 a 128 Workers) ==="
5
6 # 0. GARANTIA (Roda instantaneo se ja estiver 1/1)
7 kubectl patch deployment metrics-server -n kube-system --type='json' -p=
  ' [{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "
  value": "--kubelet-insecure-tls"} ] ' > /dev/null 2>&1
8 kubectl rollout status deployment/metrics-server -n kube-system --
  timeout=10s > /dev/null 2>&1
9
10 # 1. Preparacao
11 echo ">>> Resetando arquitetura..."
12 kubectl delete -f oran-architecture.yaml --ignore-not-found=true > /dev/
  null 2>&1
13 kubectl apply -f oran-architecture.yaml
14 echo ">>> Aguardando pods subirem (20s)..."
15 sleep 20
16
17 # 3. Loop de Teste (ATUALIZADO PARA 64 e 128)
18 for WORKERS in 1 2 4 8 16 32 64 128
19 do
20     echo "-----"
21     echo ">>> Testando Nivel: $WORKERS Workers"
22
23     kubectl scale deployment benchmark-worker --replicas=$WORKERS > /dev
  /null
24     kubectl rollout status deployment/benchmark-worker --timeout=300s >
  /dev/null
25
26     echo "      === Coletando metricas (60s) ==="
27     sleep 60
28
29     # Coleta CPU/MEM (Label corrigido: app=bench-worker)
30     STATS=$(kubectl top pods -l app=bench-worker --no-headers 2>/dev/
  null | awk 'BEGIN {cpu=0; mem=0} {cpu+=$2; mem+=$3} END {print cpu
  "," mem}')
31
32     if [ -z "$STATS" ] || [ "$STATS" == "," ]; then STATS="0,0"; fi
33
34     # Coleta Latencia (Com LC_NUMERIC=C para evitar erro de virgula)
35     RAW_LOGS=$(kubectl logs -l app=ric-platform --tail=100 2>/dev/null)
36     if [ -z "$RAW_LOGS" ]; then
37         LATENCY="0"
38     else
39         LATENCY=$(echo "$RAW_LOGS" | grep "DADO_CIENTIFICO" | awk -F'
  Latencia:' '{print $2}' | awk -F'ms' '{print $1}' | LC_NUMERIC=C awk
  '{sum+=$1; cnt++} END {if (cnt > 0) printf "%.2f", sum/cnt; else

```

```
print 0}')  
40 fi  
41 if [ -z "$LATENCY" ]; then LATENCY="0"; fi  
42  
43 echo "      >>> RESULTADO: CPU/RAM: $STATS | Latencia: $LATENCY ms"  
44 echo "$WORKERS,$STATS,$LATENCY" >> resultados.csv  
45 done  
46  
47 echo "=== FIM. Dados salvos em 'resultados.csv' ==="  
48 EOF
```

E para rodá-lo, é só fazer o seguinte comando no terminal:

```
1 chmod +x benchmark_worker.sh  
2 ./benchmark_worker.sh
```

6 FASE 5: Visualização Gráfica (*Python*)

Para visualizar os dados do arquivo *CSV*, criamos um script *Python* que gera gráficos de *CPU*, Memória e Latência.

6.1 1. Instalar Bibliotecas

```
1 sudo apt update
2 sudo apt install -y python3-pip
3 pip install matplotlib pandas
```

6.2 2. Scripts de Gráficos (gerar_graficos.py)

```
1 cat <<EOF > gerar_graficos.py
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # 1. Carregar dados
6 try:
7     df = pd.read_csv('resultados.csv')
8     print("Dados carregados com sucesso.")
9     print(df) # Mostra no terminal para conferencia
10 except:
11     print("Erro: Arquivo 'resultados.csv' n o encontrado.")
12     exit()
13
14 # --- GRAFICO 1: CPU ---
15 plt.figure(figsize=(8, 6))
16 plt.plot(df['workers'], df['cpu_total_m'], marker='o', color='tab:blue',
17         linewidth=2)
18 plt.title('Consumo Total de CPU')
19 plt.xlabel('N mero de Workers')
20 plt.ylabel('CPU (mCore)')
21 plt.grid(True, linestyle='--')
22 plt.savefig('grafico_cpu.png')
23 plt.close()
24 print(">>> Salvo: grafico_cpu.png")
25
26 # --- GRAFICO 2: RAM ---
27 plt.figure(figsize=(8, 6))
28 plt.plot(df['workers'], df['ram_total_mi'], marker='s', color='tab:green',
29         linewidth=2)
30 plt.title('Consumo Total de RAM')
31 plt.xlabel('N mero de Workers')
32 plt.ylabel('Mem ria (MiB)')
33 plt.grid(True, linestyle='--')
34 plt.savefig('grafico_ram.png')
35 plt.close()
36 print(">>> Salvo: grafico_ram.png")
37
38 # --- GRAFICO 3: LATENCIA ---
39 plt.figure(figsize=(8, 6))
40 plt.plot(df['workers'], df['latencia_media_ms'], marker='^', color='tab:red',
41         linewidth=2)
42 plt.title('Lat ncia M dia (RTT)')
```

```
40 plt.xlabel('N mero de Workers')
41 plt.ylabel('Tempo (ms)')
42 plt.grid(True, linestyle='--')
43 plt.savefig('grafico_latencia.png')
44 plt.close()
45 print(">>> Salvo: grafico_latencia.png")
46
47 print("\n=== Todos os graficos foram gerados separadamente! ===")
48 EOF
```

7 Rotina Diária de Execução

Siga esta ordem para garantir sucesso.

7.1 Passo 1: Ligar o *Docker*

```
1 sudo systemctl start docker
```

7.2 Passo 2: Preparar o *Cluster*

```
1 kubectl get nodes
2 # Se falhar: kind create cluster --name oran-lab
```

7.3 Passo 3: Verificar o Status Geral

Aqui você descobre se o *Docker* e o *Kind* estão vivos.

```
1 docker ps # Deve listar containers
2 kind get clusters # Deve mostrar 'oran-lab'
3 kubectl get nodes # Deve mostrar status 'Ready'
4 # Se o kind nao mostrar cluster, rode:
5 # kind create cluster --name oran-lab
```

7.4 Passo 4: Atualizar o Código no *Kubernetes*

```
1 cd ~/Downloads/ORAN_LAB
2
3 # A. Constroi a imagem
4 docker build -t oran-lab:v1 -f Dockerfile .
5
6 # B. Envia para o Kind
7 kind load docker-image oran-lab:v1 --name oran-lab
8
9 # C. Roda a arquitetura, caso precise
10 kubectl apply -f oran-architecture.yaml
11
12 # D. Reseta os pods para pegar a versao nova
13 kubectl delete -f oran-architecture.yaml --ignore-not-found=true
```

7.5 Passo 5: Rodar o Benchmark

```
1 chmod +x benchmark_worker.sh
2 ./benchmark_worker.sh
```

7.6 Passo 6: Gerar os Gráficos

```
1 python3 gerar_graficos.py
```


8 Referências

Para aprofundar os conceitos de *O-RAN*, *RIC*, *xApps* e *rApps*, bem como o uso de *Kubernetes* com *Kind*, consulte as seguintes fontes:

- O-RAN ALLIANCE. *O-RAN Specifications*. Disponível em: <https://www.o-ran.org/specifications>. Acesso em: nov. 2025.
- M. Polese et al. *Understanding O-RAN: Architecture, Interfaces, Algorithms, Security and Research Challenges*. IEEE, 2023.
- J. F. Santos et al. *Managing O-RAN Networks: xApp Development from Zero to Hero*. arXiv, 2024. Disponível em: <https://arxiv.org/abs/2407.09619>. Acesso em: dez. 2025.
- A. Huff et al. *A Tutorial on O-RAN Deployment Solutions for 5G: From Simulation to Real-World Testbeds*. IEEE Communications Surveys & Tutorials, 2024.
- Rimedo Labs. *RAN Intelligent Controller (RIC): Overview, xApps, and rApps*. Disponível em: <https://rimedolabs.com/blog/ran-intelligent-controller-ric-overview-xapps-and-rapps/>. Acesso em: nov. 2025.
- Accelleran. *An xApp and an rApp – What’s the difference and what do they do?*. Disponível em: <https://accelleran.com/xapp-rapp-2/>. Acesso em: dez. 2025.
- STL Partners. *RIC, xApps, rApps: A Comprehensive Guide to the Key Players*. 2024. Disponível em: <https://stlpartners.com/articles/network-innovation/ric-xapps-rapps-who-are-the-key-players/>. Acesso em: dez. 2025.
- srsRAN Project. *O-RAN NearRT-RIC and xApp*. Documentação. Disponível em: <https://docs.srsran.com/projects/project/en/latest/tutorials/source/near-rt-ric/source/index.html>. Acesso em: dez. 2025.
- Open AI Cellular. *Developing a custom xApp in Python*. Disponível em: https://openaicellular.github.io/oaic/xapp_python.html. Acesso em: dez. 2025.
- O-RAN Software Community. *RIC xApp Framework (ric-plt-xapp-frame)*. Disponível em: <https://github.com/o-ran-sc/ric-plt-xapp-frame>. Acesso em: dez. 2025.
- Kubernetes-sigs. *kind - Kubernetes IN Docker*. Repositório oficial. Disponível em: <https://github.com/kubernetes-sigs/kind>. Acesso em: nov. 2025.
- Emerson Silva. *Kubernetes — Criando um cluster local com Kind*. Blog, 2025. Disponível em: <https://emerson-silva.blog.br/posts/construindo-um-cluster-k8s-kind/>. Acesso em: dez. 2025.
- U.S. NTIA. *Open RAN Security Report*. 2022. Disponível em: https://www.ntia.gov/sites/default/files/publications/open_ran_security_report_full_report_0.pdf. Acesso em: dez. 2025.
- L. Bonati et al. *Towards Autonomous Open Radio Access Networks*. Relatório técnico, ITU. Disponível em: <https://www.paul-harvey.org/www.itu.int/pub/towards-autonomous-oran.pdf>. Acesso em: dez. 2025.

- Bubbleran. *xApps, rApps and Agentic AI: The Brains Behind RAN Automation*. 2025. Disponível em: <https://bubbleran.com/news/xapps-rapps/>. Acesso em: dez. 2025.